

PyScript

Compilador académico de lenguaje tipado estático con destino a JavaScript

Análisis léxico · Análisis sintáctico (AST) · Análisis semántico y tabla de símbolos

25 de septiembre de 2026

FLUJO

1

Léxico

Fuente → Tokens

2

Sintáctico

Tokens → AST

3

Semántico

AST + Símbolos

→

Destino

JavaScript

Características generales

- **Estructura obligatoria:** `program Nombre { ... }`
- Toda sentencia termina con `;`
- Tipado explícito y estático: `int`, `float`, `bool`, `string`
- Colecciones tipadas: `int[]`, `stack<T>`, `queue<T>`
- `int x = 10;` es válido — `x = "hola";` es error semántico
- Implementación de constantes inmutables

```
program Main {  
    const int LIMIT = 5;  
    int age = 20;  
    int[] nums = [10, 20, 30];  
    stack<int> pila;  
    if (age >= 18) {  
        print("Adulto");  
    }  
}
```

FLUJO DEL ANALIZADOR

1 LÉXICO

lexer.ts

Fuente → Token[]
línea + columna

2 SINTÁCTICO

parser.ts + ast.ts

Token[] → AST
descenso recursivo

3 SEMÁNTICO

semantic + tabla

AST → AST tipado
scopes y tipos

EXTRA

INTERFÁZ WEB

MUESTRA EN TIEMPO REAL LOS
TOKENS, AST Y TABLA DE
SIMBOLOS

PLAYGROUND Y EDITOR DE CÓDIGO

CARGA DE CÓDIGOS EJEMPLO Y DE
EDICIÓN DE LOS MISMOS

MODO PASO A PASO

EJECUCIÓN PASO A PASO,
INDICANDO CUANDO ENCUENRA
ERRORES

Del texto a los tokens

- Scanner con maximal munch: $\geq$ es un token, no $> + =$
- 46 tokens: 15 reservadas, literales, operadores, delimitadores + EOF
- Cada token guarda tipo, lexema, línea y columna
- Ignora espacios y comentarios //
- Errores: @ inesperado, string sin cierre, float 10. incompleto

```
// int age = 20;
INT          "int"
IDENT        "age"
ASSIGN       "="
INT_LIT      "20"
SEMICOLON    ";"
stack<int> → STACK < INT >
ERROR @ línea 1, col 11
```

Lista de tokens definidos

- Reservadas: program, int, float, bool, string, stack, queue, if, else, for, while, const, print, read, true, false
- Identificadores: [A-Za-z_][A-Za-z0-9]* — case-sensitive, sin colisionar con reservadas
- Literales: integer 20 · float 3.14 · string "hola" · boolean true / false
- Operadores (15) y delimitadores (9): detalle en la tabla de la derecha
- Especiales: EOF al fin de archivo · ERROR con recuperación ante @, # o string sin cierre

```
// OPERADORES (15) - lexer.ts
+ → PLUS      - → MINUS
* → MULT      / → DIV      % → MOD
= → ASSIGN
== → EQUAL_EQUAL  != → NOT_EQUAL
< → LESS      <= → LESS_EQUAL
> → GREATER   >= → GREATER_EQUAL
&& → AND      || → OR      ! → NOT
// DELIMITADORES (9)
( ) → PAREN   { } → BRACE
[ ] → BRACKET ; → SEMICOLON
, → COMMA    . → DOT
// Total: 46 tokens + EOF
```

Gramática. Ejemplos

```
program ::= "program" IDENTIFIER "{" statementList "}" EOF ;
```

- `||` → `logicalOrExpr`
- `&&` → `logicalAndExpr`
- `==, !=` → `equalityExpr`
- `<, <=, >, >=` → `relationalExpr`
- `+, -` → `additiveExpr`
- `*, /, %` → `multiplicativeExpr`
- Unarios `!, - (der.→izq.)` → `unaryExpr`
- Primarios, `()`, `a[i]`, `obj.met()` → `primaryExpr`
- Parser: descenso recursivo, un método por regla

```
if ::= "if" "(" expr ")" block [ "else" ( ifStatement | block ) ]  
· for ::= "for" "(" init ";"  
expr ";" update ")" block
```

El programa como árbol tipado

- Nodos sentencia: VarDecl, ConstDecl, If, For, While, print, read, Assignment
- Nodos expresión: Binary, Unary, Literal, Identifier, ArrayLiteral, MethodCall
- For guarda init, condition, update y body por separado

```
Program: Main
├─ VarDecl int age = 20
├─ IfStatement (age >= 18)
│   ├── then: print("Adulto")
│   └─ else: print("Menor")
└─ ForStatement
    ├── init: int i = 0
    ├── cond: i < LIMIT
    └─ body: print(nums[i])
```

¿Tiene sentido el programa?



Condición bool

if / for / while
exigen tipo bool



Const inmutable

const exige valor;
read(x) solo si mutable



Tipos compatibles

+ - * / % numéricos;
&& || ! solo bool



Colecciones

elementos homogéneos;
índice int; push/pop
validados

visitIfStatement · visitForStatement · visitWhileStatement · visitMethodCall — cada visita valida y abre ámbito propio

```
MAX = 200;           // ERROR: constante inmutable
x = "hola"; // ERROR: int ← string incompatible
if (x) { ... }       // ERROR: la condición no es bool
```

Ámbitos y aislamiento

- Bloques de código: `enterScope()` al entrar a `{ }`, `exitScope()` al salir
- Cada símbolo: `name`, `type`, `kind`, `scope`, `mutable`
- `lookup()` busca de adentro hacia afuera → permite shadowing (acceder a variables fuera del Bloque donde se declararon)
- Scope isolation: lo declarado en un bloque muere con el bloque

EJEMPLO DE AISLAMIENTO

```
if (true) {  
    int temp = 42;  
}  
print(temp); // ERROR: temp no existe  
aquí
```

SÍMBOLOS REGISTRADOS

```
age : int · variable · scope 0 · mutable  
MAX : int · constant · scope 0 ·  
inmutable  
pila : stack<int> · variable · scope 1
```

Reglas de transición $Q1 \rightarrow Q2$

Q1 estado inicial · Q2 estado final · [V]/[F] condición · ^ conjunción AND · (S1)/(S2) ramas · (INIT)/(UPDT) cabecera de ciclo

IF SIMPLE

$Q1 \rightarrow \text{COND } [V] \rightarrow (S1) \rightarrow Q2$
 $Q1 \rightarrow \text{COND } [F] \rightarrow Q2$

if (a && b) exige $[V] \wedge [V]$

IF – ELSE

$Q1 \rightarrow \text{COND } [V] \rightarrow (S1) \rightarrow Q2$
 $Q1 \rightarrow \text{COND } [F] \rightarrow (S2) \rightarrow Q2$

else if = if anidado en (S2)

WHILE

$Q1 \rightarrow \text{COND } [V] \rightarrow (S1) \rightarrow Q1 \dots$
 $Q1 \rightarrow \text{COND } [F] \rightarrow Q2$

cero iteraciones si [F] inicial

FOR

$Q1 \rightarrow (\text{INIT}) \rightarrow \text{COND } [V] \rightarrow$
 $(S1) \rightarrow (\text{UPDT}) \rightarrow Q1 \dots \rightarrow Q2$

cabecera: init ; cond ; update

Un error por fase, con línea y columna

LÉXICOS

lexer.ts

```
int x = 10 @ 5;
```

```
"Cesar sin cierre
```

```
float f = 10.;
```

SINTÁCTICOS

parser.ts

```
int x = ;
```

```
if (x > 5) print(x);
```

```
for (int i = 0 i < 5)
```

SEMÁNTICOS

semantic

```
y = 20; // no declarada
```

```
MAX = 200; // const
```

```
if (x) // exige bool
```

Alcance v0.1 y trabajo futuro

ALCANCE Y LÍMITES v0.1

- Sin funciones de usuario, clases, módulos ni excepciones
- const solo primitivos con valor obligatorio;
- a[i] un nivel
- for con init primitiva; condiciones siempre bool

TRABAJO FUTURO

- Optimizador: constant folding + dead code elimination
- Generador JS + CLI: `pyscript main.pys -o main.js`

El Playground web (React + D3.js) existe como herramienta de depuración fase por fase, pero el corazón del proyecto son los analizadores.